

optima-documents

Kgoana K.B Mametja



Optima code documentation
12 June 2026
Overview of the codes used for
technical analysis of JSE assets

Executive Summary

During the first week of the **JSE investment challenge** the team experimented with different methodologies of technical stock research. After a week of development and testing, the optimal option was found.

This document is an in-depth step by step explanation of the method used to assess if an asset is a worthy investment. It will start with the first action within the methodology followed by the next with explanations and reasons.

Prerequisites

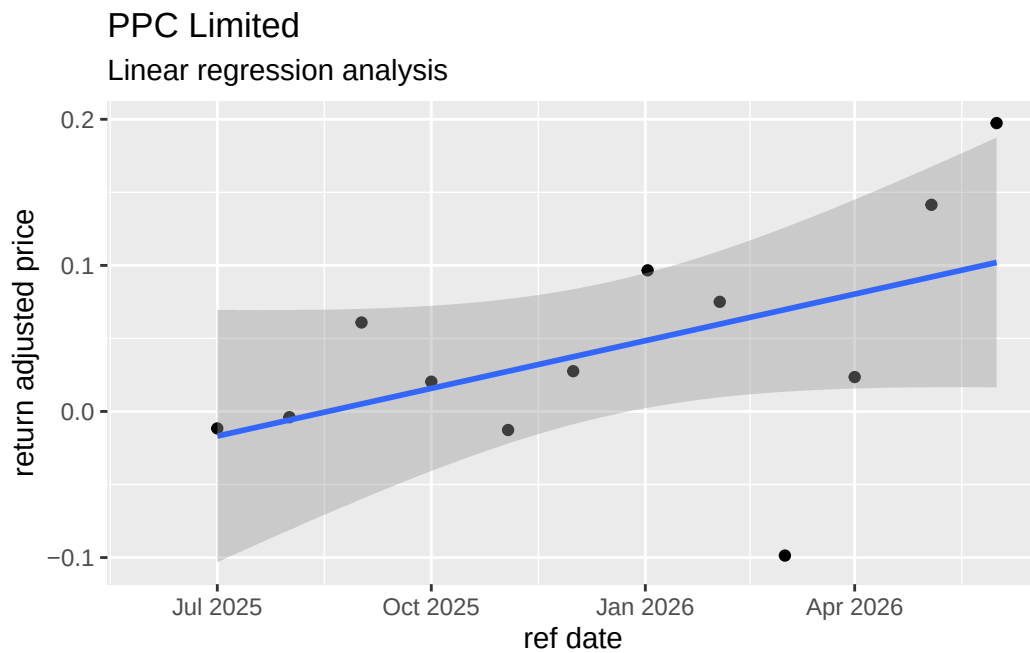
- R studio
- List of tickers to investigate
- Libraries - ggplot2, quantmod, tidyverse, PerformanceAnalytics

Method

1. Linear regression filter

```
library(yfR)
library(ggplot2)
library(tidyverse)

df <- yf_get(tickers = 'PPC.JO', first_date = '2025-06-01', last_date = Sys.Date(), freq_data = 'daily')
df_plot <- df %>% select(ref_date, ret_adjusted_prices)
ggplot(df_plot, aes(x = ref_date, y = ret_adjusted_prices))+
  geom_point()+
  geom_smooth(method = 'lm', se = TRUE)+
  labs(title = 'PPC Limited',
       subtitle = 'Linear regression analysis',
       x = 'ref date',
       y = 'return adjusted price')
```



Using Linear Regression as a Stock Filter (Team Reference)

Optima uses linear regression to identify whether a stock has a consistent positive trend over time. The regression prediction shows the expected value based on historical data, while the prediction error indicates how closely the stock follows that trend.

Because the team is in a competition, only stocks with a **positive trend** are selected for further investigation. Stocks with a flat or negative trend are filtered out immediately.

From the example provided, the predictions increase steadily year over year (from <0.0 to >0.1) this represents a clean, positive trend that would qualify a stock for deeper analysis.

In practice:

- **Positive trend** → Flag the stock for further research.
- **Flat or negative trend** → Discard, no further investigation needed.

This approach allows the team to focus only on stocks moving in a favorable direction, saving time and concentrating effort on opportunities with the highest potential in the competition.

2. Price Return Analysis

If the asset shows a positive trend during the linear regression analysis, the methodology will then move onto the second part of the research which is the price return analysis.

Plotting the return adjusted prices accounts for:

- Dividend distributions
- Stock splits
- Corporate actions

This is a preliminary screening tool for asset price history—useful for spotting obvious trends/volatility, but should be combined with fundamental analysis and sector comparisons for investment decisions.

Plotting the return adjusted prices reveal:

- **Trend direction** (bullish/bearish phases)
- **Support/resistance levels** (psychological price barriers)
- **Volatility clustering** (risk assessment)
- **Seasonal patterns** (e.g., construction sector cycles in South Africa)

```
library(yfR)
library(ggplot2)
library(tidyverse)

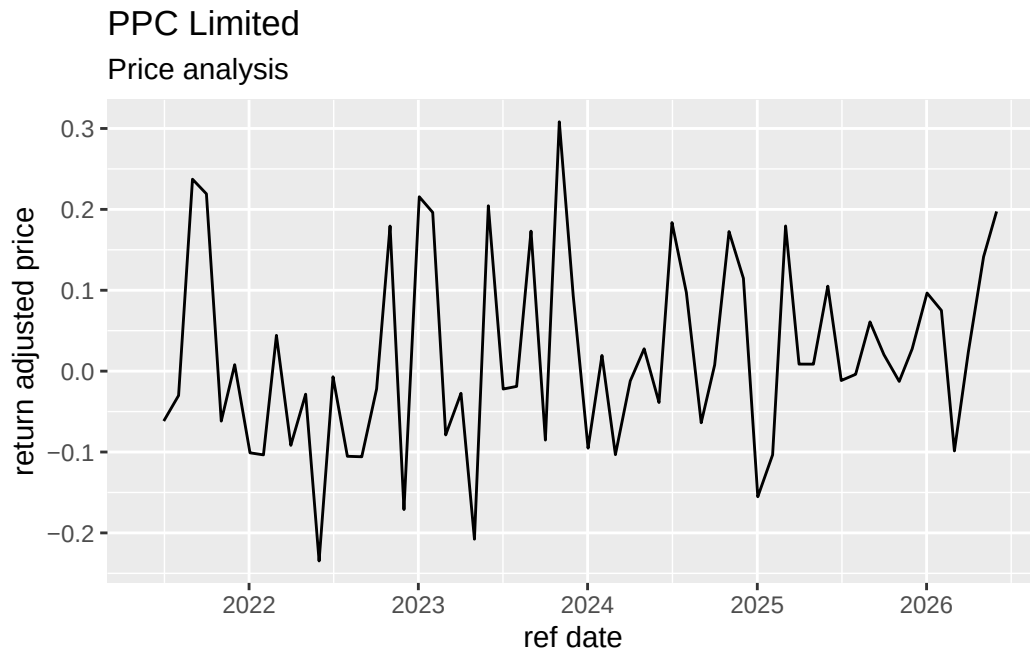
df <- yf_get(tickers = 'PPC.JO', first_date = '2021-06-01', last_date = Sys.Date(), freq_data = 'daily')
summary(df)
```

	ticker	ref_date	volume	price_open
Length	:61	Min. :2021-06-01	Min. : 20659291	Min. :216.0
N.unique	: 1	1st Qu.:2022-09-01	1st Qu.: 37607948	1st Qu.:304.0
N.blank	: 0	Median :2023-12-01	Median : 54950995	Median :393.0
Min.nchar	: 6	Mean :2023-12-01	Mean : 70916511	Mean :406.5
Max.nchar	: 6	3rd Qu.:2025-03-03	3rd Qu.: 78503766	3rd Qu.:505.0
		Max. :2026-06-01	Max. :381341613	Max. :764.0

	price_high	price_low	price_close	price_adjusted
Min.	:238.0	Min. :186.0	Min. :218.0	Min. :217.7
1st Qu.	:359.0	1st Qu.:264.0	1st Qu.:305.0	1st Qu.:304.5
Median	:418.0	Median :346.0	Median :390.0	Median :389.9
Mean	:446.5	Mean :366.5	Mean :409.3	Mean :409.0
3rd Qu.	:534.0	3rd Qu.:449.0	3rd Qu.:505.0	3rd Qu.:504.2
Max.	:834.0	Max. :642.0	Max. :831.0	Max. :831.0

	ret_adjusted_prices	ret_closing_prices	cumret_adjusted_prices
Min.	:-0.234667	Min. : -0.234667	Min. :0.5604
1st Qu.	:-0.067594	1st Qu.: -0.068161	1st Qu.:0.7841
Median	: 0.001879	Median : 0.001879	Median :1.0037
Mean	: 0.019715	Mean : 0.019690	Mean :1.0530
3rd Qu.	: 0.099101	3rd Qu.: 0.099010	3rd Qu.:1.2982
Max.	: 0.308244	Max. : 0.308244	Max. :2.1395
NAs	:1	NAs :1	

```
df_plot <- df %>% select(ref_date, ret_adjusted_prices)
ggplot(df_plot, aes(x = ref_date, y = ret_adjusted_prices))+
  geom_line()+
  labs(title = 'PPC Limited',
        subtitle = 'Price analysis',
        x = 'ref date',
        y = 'return adjusted price')
```



3. Getting prices and returns in quantmod

```
library(TTR)
library(yfR)
library(ggplot2)
library(quantmod)
```

Loading required package: xts

Loading required package: zoo

Attaching package: 'zoo'

The following objects are masked from 'package:base':

as.Date, as.Date.numeric

```
##### Warning from 'xts' package #####
#
# The dplyr lag() function breaks how base R's lag() function is supposed to #
# work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or #
# source() into this session won't work correctly. #
#
# Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
# conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop #
# dplyr from breaking base R's lag() function. #
#
# Code in packages is not affected. It's protected by R's namespace mechanism #
# Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning. #
#
#####
```

Attaching package: 'xts'

The following objects are masked from 'package:dplyr':

first, last

```
library(tidyverse)
library(PerformanceAnalytics)
```

Attaching package: 'PerformanceAnalytics'

The following object is masked from 'package:graphics':

legend

```
library(quantmod)

ticker <- "PPC.JO"

data <- getSymbols(ticker, src = "yahoo",
                  from = "2025-01-01",
                  auto.assign = FALSE)
```

```
#prices
price <- Cl(data)

#returns
returns <- dailyReturn(Cl(data))
```

This is where the real in-depth analysis begins, the closing prices and daily returns based on the closing prices of the dataframes are extracted into their own individual dataframes to be used later for the analysis.

Momentum

```
# momentum
price_vec <- na.omit(as.numeric(price))

momentum_1m <- (last(price_vec) / first(tail(price_vec, 21)) - 1) * 100
momentum_3m <- (last(price_vec) / first(tail(price_vec, 63)) - 1) * 100
momentum_6m <- (last(price_vec) / first(tail(price_vec, 126)) - 1) * 100
```

values	
momentum_1m	24.4011976047904
momentum_3m	44.7735191637631
momentum_6m	59.5009596928983

Figure 1: Momentum output

This code takes the price dataframe, turns it into a numeric datatype, and omits any null values and saves the transformed dataframe as the price vector. The formula for momentum is:

$$\text{Momentum (\%)} = (\text{Current Price} / \text{Price N days ago} - 1) \times 100$$

Momentum measures **the rate of price change** over a specific period — essentially, how fast and in which direction the price has been moving, whether or not it moves in a positive or negative direction.

Signal

```
ma20 <- SMA(price, 20)
ma50 <- SMA(price, 50)

ma20_clean <- na.omit(ma20)
ma50_clean <- na.omit(ma50)
price_clean <- na.omit(price)

signal <- ifelse(
  as.numeric(last(price_clean)) > as.numeric(last(ma20_clean)) &
  as.numeric(last(ma20_clean)) > as.numeric(last(ma50_clean)),
  "Bullish",
  "Bearish"
)
```

Calculates 20-day and 50-day **Simple Moving Averages (SMA)**, then generates a **trend signal** (Bullish/Bearish) based on the current price position relative to these moving averages.

A bullish signal occurs when the current price is above both the 1-month and 2.5-month moving averages and the 1-month average is above the 2.5-month average — confirming the uptrend is established.

RSI (Relative Strength Index)

RSI measures the **magnitude of recent price changes** to evaluate whether a stock is **overbought** or **oversold**. It quantifies the **peed and velocity** of price movements on a scale of **0 to 100**.

$$RSI = 100 - [100 / (1 + RS)]$$

Where: $RS = (\text{Average Gain over } N \text{ periods}) / (\text{Average Loss over } N \text{ periods})$

```
rsi <- RSI(price, n = 14)
tail(rsi, 1)
```

```
      rsi
2026-06-12 82.88015
```

It determines if the asset is overbought (> 70, strong), 50-70 (healthy), Oversold (< 30 unhealthy).

Volatility

```
volatility <- sd(returns) * sqrt(252)
```

Calculates the **annualized volatility** of the stock returns — a measure of how much the stock price fluctuates on an annual basis.

We use 252 because its an approximate number of trading days (365 days - 104 weekend days - ~9 holidays)

Meaning if:

Daily returns: +1%, -0.5%, +2%, -1%, +1.5%

Mean return: +0.6%

`sd(returns) = 1.2%` ← Typical daily move is $\pm 1.2\%$

If the sd is 1.2%, it means the returns **typically** fluctuate up OR down by about 1.2% from the average return.

Sharpe-ratio

```
# sharp-ratio
sharpe <- SharpeRatio.annualized(returns)
sharpe_value <- as.numeric(sharpe)
```

Calculates the **annualized Sharpe Ratio** — a measure of **risk-adjusted return** that tells how much excess return an investor getting for each unit of risk (volatility) you take.

This is usually best when comparing different assets rather than an individual asset. For example if:

Stock A:

- Annual return: 15%
- Risk-free rate: 8% (South African government bond)
- Volatility: 20%

$$\text{Sharpe} = (15\% - 8\%) / 20\% = 7\% / 20\% = 0.35$$

Stock B:

- Annual return: 12%
- Risk-free rate: 8%
- Volatility: 10%

$$\text{Sharpe} = (12\% - 8\%) / 10\% = 4\% / 10\% = 0.40$$

The greater the Sharpe-ratio the better the investment, meaning Stock B is the best investment because lower it has lower returns, it achieves this with less risk.

To understand better interpret the Sharpe ratio better, the table below provides more information:

SHARPE RATIO	INTERPRETATION
< 0	Poor (investment underperformed the risk-free rate)
0 – 0.5	Weak
0.5 – 1.0	Acceptable/Fair
1.0 – 2.0	Good
2.0 – 3.0	Very Good
> 3.0	Excellent

Scoring Model

```
# score model
momentum_norm <- as.numeric(momentum_3m / 100)
rsi_value <- as.numeric(last(rsi))
rsi_norm <- (rsi_value - 50) / 50
signal_value <- ifelse(signal == "Bullish", 1, 0)

score <- 0.4 * momentum_norm +
  0.3 * sharpe_value +
  0.2 * rsi_norm +
  0.1 * signal_value
```

Creates a **weighted composite score** that combines four different indicators into a single numerical value, normalized to be comparable across different metrics.

The transformation:

- We divide the momentum by 100 to convert it to a decimal form.
- We divide and subtract RSI by 50 if we subtract 50 it now runs from -50 to +50 and if we divide by 50 it now runs from -1 to +1.
- Signal is binary so no need for transformation.
- The weighting scheme is

$$\text{Score} = 40\% \text{ Momentum} + 30\% \text{ Sharpe} + 20\% \text{ RSI} + 10\% \text{ Trend Signal}$$

Scoring interpretation:

Score Range	In Simple Words
> 0.5	Strong Buy — great stock
0.25 - 0.5	Buy — good stock
0 - 0.25	Hold — wait and see
-0.25 - 0	Weak — might want to sell
< -0.25	Sell — bad stock, get out

Conclusion

This is a document explaining the technical methodology of the Optima teams approach to researching assets and stocks.